

中山大学 计算机学院本科生实验报告

(2025/2026学年)

课程名称	并行程序设计与算法 (实验)	批改人	
实验	Lab 2 - MPI 集合通信与并行矩阵乘法	专业 (方向)	计算机科学与技术 (人工智能)
学号	23336103	姓名	雷颜玮
Email	leiyanyanwei2005@163.com	完成日期	2026年4月17日

1. 实验要求与目的

- 改进上次实验中的 MPI 并行矩阵乘法 (MPI-v1), 采用 **MPI 集合通信** 实现进程间通信。
- 使用 `MPI_Type_create_struct` 聚合 MPI 进程内变量 (m, n, k) 后进行通信。
- 调整并记录不同进程数量 (1_{16}) 及矩阵规模 (128^{2048}) 下的时间开销, 填写表格, 并分析其性能及扩展性。
- 探讨不同通信方式 (集合通信 vs 点对点通信) 对性能的影响, 并应用 Cache 局部性优化。

2. 实验过程与核心代码

2.1 聚合通信变量机制

为了避免使用多次独立的 `MPI_Bcast` 或者强行将不同变量塞入同一个数组进行传递, 本实验定义了 `MatrixConfig` 结构体, 并通过 `MPI_Type_create_struct` 将其注册为 MPI 识别的自定义数据类型, 从而实现一次性的广播, 降低了通信延迟。

```
// 定义用来全员广播的结构体
struct MatrixConfig { int m; int n; int k; };

// 创建并提交 MPI 自定义结构体类型
MPI_Datatype mpi_config_type;
int blocklengths[3] = {1, 1, 1};
MPI_Aint displacements[3] = {offsetof(MatrixConfig, m), offsetof(MatrixConfig, n), offsetof(MatrixConfig, k)};
MPI_Datatype types[3] = {MPI_INT, MPI_INT, MPI_INT};

MPI_Type_create_struct(3, blocklengths, displacements, types, &mpi_config_type);
MPI_Type_commit(&mpi_config_type);

// 使用集合通信, 一次性将 m, n, k 广播给所有进程
MPI_Bcast(&config, 1, mpi_config_type, 0, MPI_COMM_WORLD);
MPI_Type_free(&mpi_config_type);
```

2.2 集合通信：任务分发与结果收集

摒弃了 Lab1 中 Master 进程通过 `for` 循环逐一 `MPI_Send` 的低效做法，本实验全面接入集合通信：

- **广播矩阵 B**：使用 `MPI_Bcast` 将只读的 B 矩阵一次性全员广播。
- **分发矩阵 A**：使用 `MPI_Scatter` 自动将矩阵 A 切割并均匀散发到各工作进程中。
- **收集矩阵 C**：使用 `MPI_Gather` 将各进程计算好的局部结果 C 收集并无缝拼接到 0 号进程的完整数组中。

```
// 1. 广播完整的矩阵 B 给所有人
MPI_Bcast(B.data(), n * k, MPI_DOUBLE, 0, MPI_COMM_WORLD);

// 2. 切块平分矩阵 A 到所有人的 local_A 里
MPI_Scatter(full_A.data(), local_m * n, MPI_DOUBLE,
            local_A.data(), local_m * n, MPI_DOUBLE, 0, MPI_COMM_WORLD);

// ... 局部矩阵乘法计算 ...

// 3. 将所有人计算好的 local_C 拼接回 0 号进程的 full_C 中
MPI_Gather(local_C.data(), local_m * k, MPI_DOUBLE,
            full_C.data(), local_m * k, MPI_DOUBLE, 0, MPI_COMM_WORLD);
```

2.3 软件缓存 (Cache) 局部性优化

在子进程局部的核心矩阵乘法计算中，将常规代码里的 `i-j-p` 层级重排改写为 `i-p-j` 循环。此举使得最内层寻址沿着一维数组连续执行，充分利用了现代 CPU 的 Cache 预取机制。

```
for (int i = 0; i < local_m; ++i) {
    for (int p = 0; p < n; ++p) {
        for (int j = 0; j < k; ++j) {
            // j 在最内层递增，与 C++ 行优先连续存储特征完美匹配
            local_C[i * k + j] += local_A[i * n + p] * B[p * k + j];
        }
    }
}
```

3. 性能测试数据

根据自动化测试跑出的真实输出数据记录如下（单位：秒）：

进程数 \ 矩阵规模	128	256	512	1024	2048
1	0.000668	0.003785	0.022103	0.194725	3.256670
2	0.000251	0.001650	0.011619	0.121018	1.691910
4	0.000149	0.001303	0.007577	0.101047	1.229250
8	0.000170	0.000751	0.004462	0.089935	0.940404
16	0.000222	0.000657	0.005400	0.097003	0.984881

(注：由于物理核心限制，测试 16 进程时使用了 `--oversubscribe` 超额订阅，频繁的线程上下文切换不可避免地导致了性能下降，属硬件限制引起的正常现象。)

4. 核心技术讨论与分析

4.1 MPI 集合通信 vs 点对点通信

在 Lab1 中，使用点对点通信分发任务需要 0 号进程通过 `for` 循环逐个向所有工作进程发送片段。当节点骤增时，0 号进程将成为网络带宽的单点瓶颈。而在 Lab2 中采用集合通信（如 `MPI_Bcast` 和 `MPI_Scatter`），底层大多通过“二叉树拓扑”等优越算法将通信分发压力平摊到了沿途节点，大幅提升了多进通信吞吐上限；同时彻底避免了大量手动编写 `MPI_Send/Recv` 导致的繁杂边界判定逻辑。

4.2 MPI_Type_create_struct 聚合变量的重要性

矩阵维度 `m, n, k` 单独只有几个字节，如果强制调用 3 次 `MPI_Bcast` 下发，将引发极大的握手通信延迟 (Latency) 开销。将其打包封装为 `struct` 并调用 `MPI_Type_create_struct` 构建对应类型，实现了仅仅发生 1 次网络往返即可完成轻量异构数据的传递，符合并行网络通信对于“合并琐碎消息”的黄金准则。

4.3 Cache 局部性 (i-p-j 重排) 优化原理

C++ 一维模拟数组完全由行优先 (Row-major) 在内存上连续堆叠。如果是 `i-j-p` 循环，最内层计算 `B[p, j]` 的迭代指针为 `p`，每次自增都必须跳越一整行去取下一个元素，其严重违背了内存空间局部性，使得 L1/L2 Cache 各级缓存发生巨量 Miss 浪费；重排为 `i-p-j` 顺序后，最内层计算步长为 `j`，此时 `B[p, j]` 和 `local_C[i, j]` 全部变为紧凑连续的内存顺次游历读取，无需额外空间计算代价，仅凭 Cache 命中率即可让最终性能数倍暴增。

4.4 规模的扩展性 (Scalability) 分析

从实验表格中我们能够观测出：对 128~512 的小规模矩阵，盲目拉起过多进程往往由于 **MPI 进程创建建立以及分发切片时的内部通信耗时** 构成了主导大头，进而体现出加速效率折损（例如小矩阵在 8/16 进程耗时甚至不如单进程）。而在矩阵规模成倍剧增突破 1024~2048 后，高额纯算术开销压倒了一切系统管理开销，多核加速表现得非常优异，且规模越大由于通讯所占相对比例的摊薄，总时间加速比越趋向于理想的准线性正比增长。